

Rapport projet Java

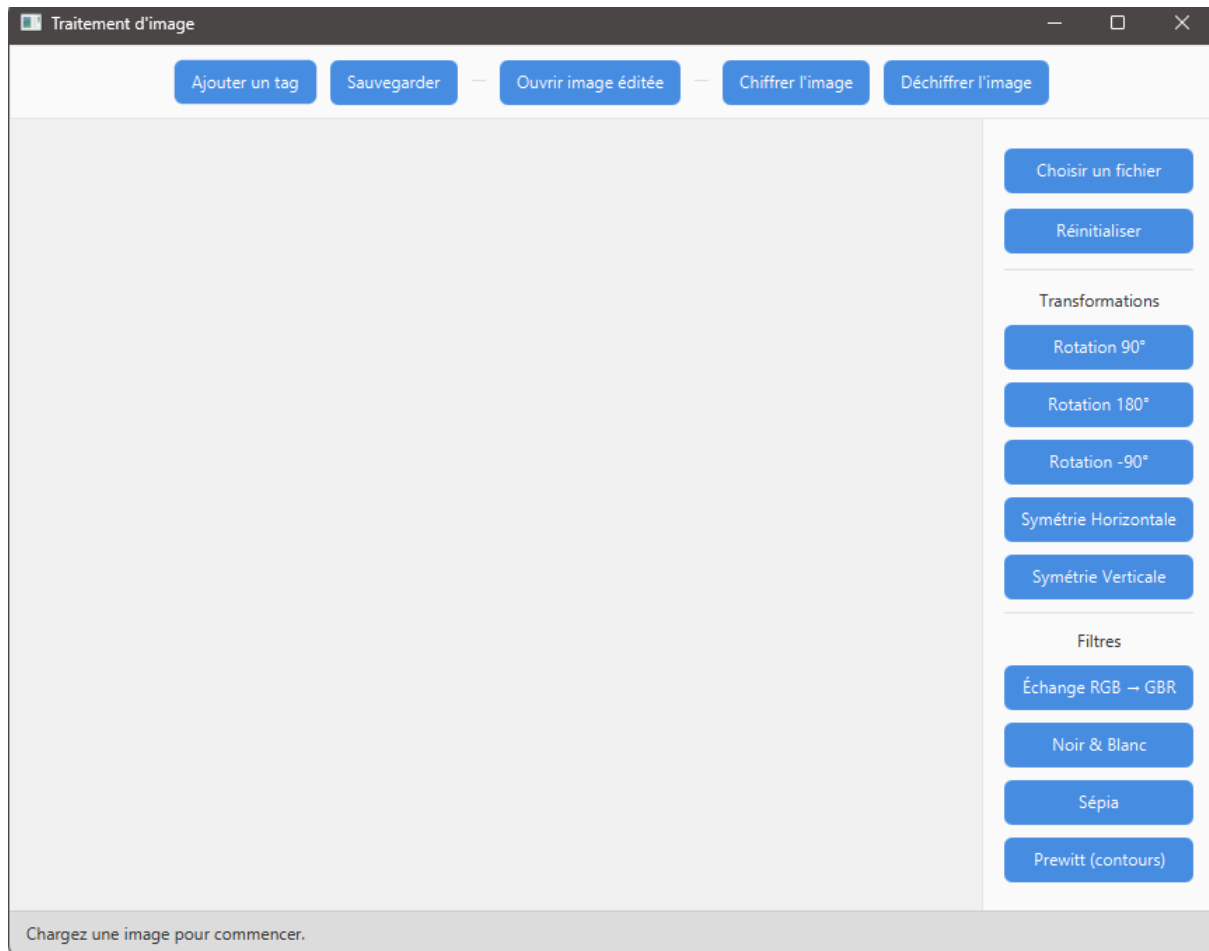
Année universitaire 2025/2026

Par Wassim Aichaoui et Laura Deruwez

Introduction

Ce projet consistait à faire une application qui permet un traitement d'image en Java, en utilisant Javafx et en ayant une architecture MVC grâce à l'utilisation de fxml.

Voici un aperçu de l'application quand on l'ouvre :



Le projet contenant beaucoup de classes, pour rendre la compréhension visuelle et ainsi plus facile pour un œil extérieur, nous avons décidé de faire un [diagramme de classes](#) (.puml) à l'aide de l'IA. Dans le dossier .zip rendu, deux diagrammes sont disponibles :

- diagramme_complet.png : c'est le diagramme détaillé
- diagramme_simplifié.png : c'est le diagramme simplifié pour être plus lisible et visuel

Pour charger le projet dans IntelliJ, il faut ouvrir le pom.xml en tant que projet.

Pour lancer le projet, il faut [exécuter le fichier Launcher.java](#). Il lance lui-même ImageProcessingApp.java.

1. Structure générale du projet

a. Interface utilisateur

L'interface graphique repose sur un fichier fxml (image-app pour l'application et gallery-view pour la galerie qui sert à sélectionner une image déjà éditée). Les interactions sont gérées par les classes ImageAppController pour l'application et GalleryController pour la galerie. Ce sont ces deux classes qui gèrent le traitement des interactions avec l'utilisateur, notamment suite à l'appui d'un bouton.

b. Gestion des images

Afin d'éviter que tout ne soit géré par le contrôleur, la manipulation des images se fait par la classe ImageManager. Elle est responsable de l'image originale et l'image courante (image affichée avec les transformations), l'application des filtres, la gestion mémoire des métadonnées et la reconstruction d'une image quand on la rouvre ou la déchiffre. Il aurait peut être été plus pertinent que la gestion des métadonnées ne soit pas faite dans cette classe, pour qu'elle reste spécialisée et ne fasse pas trop de choses.

c. Traitement des images

Les transformations et les filtres de l'image ont tous un type commun (ImageOperation) grâce à l'interface. Cela permet d'utiliser l'héritage et les liaisons dynamique pour simplifier le code. On distingue deux catégories : les filtres de couleur (ImageFilter) qui contiennent les filtres sépia, prewitt, grayscale et rgbSwp, et les filtres de transformation (ImageTransform) qui gèrent les rotations et les symétries.

La classe OperationRegistry permet de retrouver dynamiquement une opération grâce à son identifiant, ce qui est très pratique pour appliquer à nouveau les transformations après un chargement.

d. Sauvegarde des données

Les données des images sont sauvegardées dans la classe ImageData, qui garde le chemin de l'image, son nom, ses tags et l'historique des opérations effectuées. La sauvegarde des métadonnées est effectuée dans un fichier JSON par la classe SaveData. La sauvegarde des images réelles (chiffrées ou non) est assurée par la classe SaveFxImage.

e. Sécurité

Deux classes gèrent la sécurité. SecurityManager existe pour faire l'intermédiaire entre le contrôleur et la classe Security qui chiffre et déchiffre les images. Cela permet que le contrôleur ne soit pas trop chargé.

Le chiffrement respecte ce qui est précisé dans l'énoncé.

2. Explications avancées

Toutes les opérations sur les images ont un type commun, ce qui permet d'appliquer les filtres avec du polymorphisme et ainsi simplifier le code. Elles transforment l'image sans connaître le reste du système, de manière indépendante. La classe ImageOperation permet d'avoir un type commun et donc d'implémenter facilement ce polymorphisme, malgré la séparation des filtres (sur les pixels) et des transformations (géométriques). Cette séparation existe car la façon d'appliquer le filtre est différente, et pour avoir une séparation claire sur l'application.

Chaque opération est identifiée par un id, qui est ensuite liée à l'objet en lui-même avec la classe OperationRegistry qui contient un dictionnaire. Cela simplifie la reconstruction des images quand elles sont chargées et rend ainsi le code plus lisible et compréhensible.

Pour permettre la persistance des données, à chaque fois qu'une opération est réalisée, elle est ajoutée à une liste de transformations de ImageData. Cette liste est ensuite enregistrée dans un JSON, puis récupérée et appliquée lors du chargement de l'image.

Le système ne stocke pas l'image finale, mais l'image originale (chiffrée ou non) et l'historique des transformations appliquées.

Le chiffrement, conformément à l'énoncé, permute les pixels de l'image de manière déterministe avec un mot de passe fourni. Seule l'image originale est chiffrée. Si l'image est sauvegardée après avoir été chiffrée, elle sera enregistrée dans la mémoire chiffrée, et non en clair.

Le projet intègre une galerie qui permet de visualiser et recharger les images précédemment sauvegardées par l'application ainsi que leurs métadonnées. L'affichage est géré par un fichier fxm, et la gestion des interactions se fait avec GalleryController. GalleryManager en contient la logique. Chaque élément contient une miniature de l'image, son nom et ses tags. Il est possible de rechercher une image par tag dans la galerie.

3. Difficultés rencontrées et choix faits

Une des premières difficulté qui a été rencontrée était au moment de recharger les transformations quand on ouvrait une image déjà sauvegardée. La classe ImageOperation qui regroupait toutes les opérations possibles n'existait pas, donc une approche avec des conditions avait été retenue provisoirement. A la fin du projet, quand tout le reste avait été fait et fonctionnait, les classes ImageOperation et OperationRegistry ont été créées et utilisées pour pallier ce problème. Cela respectait beaucoup plus le paradigme de la POO qu'un switch case !

Une autre grosse modification s'est imposée sur la fin. N'osant pas faire un second fichier fxm, la galerie était gérée directement dans une classe Java sans fxm pour gérer l'aspect graphique. Comme cela ne respectait pas les consignes, cela a été modifié à la fin du projet sur le temps restant quand tout était fini.

Une difficulté plus générale rencontrée tout au long du projet est qu'il faut savoir faire des choix et s'y tenir. Se demander vraiment ce qui est attendu et pourrait être le plus utile à l'utilisateur. Un exemple parlant peut être l'usage du bouton réinitialiser l'image. Quand on charge une image qu'on avait modifiée puis sauvegardée, l'image revient à son état d'origine avant toute modification. Deux choix s'offraient à nous :

- réinitialiser l'image totalement, ce qui peut être voulu car l'utilisateur ne veut probablement pas perdre la possibilité de revenir à avant les modifications, mais il perd la trace de quelles modifications il avait fait pour en arriver là
- réinitialiser l'image à la dernière sauvegarde, mais l'utilisateur perd la possibilité de revenir à l'image d'origine sauf si il la retrouve sur son ordinateur pour recommencer à la modifier.

Les deux choix ont leurs avantages et inconvénients, mais un seul doit être retenu. Nous avons fait le choix de réinitialiser l'image comme il était possible de revenir à la sauvegarde avec "ouvrir une image sauvegardée".

L'architecture du projet étant complexe et sa taille importante, il apparaît parfois compliqué de corriger des bogues et trouver exactement leur origine. Par exemple, même si l'image sauvegardée était chiffrée, la galerie affichait une image en clair. Il a fallu un moment avant de trouver que c'était parce qu'elle affichait le path de l'image locale à l'ordinateur et non de l'image sauvegardée dans le projet.

4. Défaits du projet et extensions possibles

La galerie d'images pour sélectionner une image sauvegardée à modifier pourrait être améliorée. Il existe une recherche par tag, mais une recherche par nom d'image pourrait être tout aussi pertinente à ajouter. De plus, les miniatures affichent l'image non modifiée et ne prennent pas en compte les transformations déjà effectuées. Il pourrait être pertinent d'effectuer les transformations sur les miniatures pour mieux permettre à l'utilisateur de choisir sur exactement quelle image il veut reprendre. Si il a plusieurs versions d'une image, cela aiderait à mieux choisir.

Le projet contient beaucoup de classes "boîte à outil" avec des méthodes statiques, comme par exemple DialogManager ou des fonctions de sauvegarde, ce qui s'éloigne un peu du paradigme orienté objet.

Vers la fin du projet, le contrôleur contenait toute la logique du code. Les classes "manager" n'existaient pas. Comme cela n'était pas optimal, cela a été changé une fois que tout fonctionnait. Nous avons essayé, autant que possible, de déléguer la logique des fonctions à des classes adaptées, que le contrôleur utilise. Malgré cela, nous trouvons que le contrôleur est encore beaucoup trop long, plus de 200 lignes, et que cela pourrait encore être amélioré.

Un ajout intéressant, pourrait être un bouton "revenir en arrière" et un bouton "revenir en avant", comme nous avons l'historique des transformations effectuées.

D'autres filtres pourraient aussi être intéressants mais plus difficiles à mettre en œuvre, comme rogner une image.